Dr. Alla Jammine
alla.jammine@u-paris.fr **Web Programming_2026**

TP_6

React Advanced & API Integration

Continuation: Task Manager → Multi-page SPA + REST API

Prerequisite: TP5 Task Manager (components + state + list + filters).
Outcome: A **multi-page React SPA** that **fetches tasks from a REST API**, supports **loading/error states**, and displays data dynamically.

Max grade: 20 points
Total time: 90 min (80 min work + 10 min submission)

What Students Will Build (Expected Result)

The final app must have **at least 3 routes**:

1. / — Home (your existing task manager UI, now with API-backed tasks)
2. **/tasks/:id** — Task Details page (loaded from API by id)
3. **/about** — About page (static content + project info)

Navigation menu must be visible on all pages.

API requirements:

- Tasks are loaded from a REST API
- UI shows:
  - **Loading** state while fetching
  - **Error** state if request fails
- Data must be displayed dynamically in components

Recommended Sample API (no backend required)

Use **JSONPlaceholder**:

- List: `https://jsonplaceholder.typicode.com/todos?_limit=15`
- Details: `https://jsonplaceholder.typicode.com/todos/{id}`

Tasks mapping:

- `id` → id
- `title` → text
- `completed` → completed

(They can keep local "add/delete" in state, but initial load must come from API.)

Dr. Alla Jammine
alla.jammine@u-paris.fr

Theoretical Background (10 min)

Students must know:

React Router (v6)

- `<BrowserRouter>`, `<Routes>`, `<Route>`
- `<Link>` for navigation
- `useParams()` for reading `:id`

Fetching data

- `fetch()` or `axios`
- `useEffect()` for side effects
- `useState()` for:
  - `data`
  - `loading`
  - `error`

Loading & Error patterns

- loading: show spinner/message
- error: show message + optional retry

Time Allocation (80 min)

Stage	Activity	Time	Expected Result
1	Install Router + create routes	10 min	App has navigation + 3 pages
2	Create API service module	10 min	`api.js` can fetch list + details
3	Fetch tasks list on Home	20 min	Tasks come from API + loading/error UI
4	Task Details route	20 min	`/tasks/:id` loads task details
5	Improve UX + finalize	20 min	Retry button, clean UI, no console errors
Submission	Demo + short check	**10 min**	Student demonstrates functionality

TP_6_Practical Assignment

Stage 1 — Router Setup (10 min)

1. Install React Router:

```
npm install react-router-dom
```

2. Create routes in `App.js`:

- `/` → Home
- `/tasks/:id` → TaskDetails
- `/about` → About

3. Add a simple navigation bar using `<Link>`.

Expected result:

- Clicking navigation changes URL and page content without reload.

Stage 2 — API Service Module (10 min)

Create `src/services/api.js` with 2 functions:

- `fetchTasks()` → list of tasks
- `fetchTaskById(id)` → one task

Expected result:

- API functions return data or throw errors properly.

Stage 3 — Fetch Tasks for Home (20 min)

On the Home page (your Task Manager screen):

- Fetch tasks on first load using `useEffect`
- Show:
  - "Loading…" while fetching
  - Error message if fetch fails

Expected result:

- Tasks displayed from API
- No infinite loops in useEffect
- No console warnings

Stage 4 — Task Details Page (20 min)

Create `/tasks/:id` page:

- Use `useParams()` to read id

- Fetch single task by id
- Show loading/error states
- Display:
  - id
  - title
  - completed status

Also: Make each task on Home clickable:

- clicking a task navigates to `/tasks/{id}`

Expected result:

- Details page loads correct task.

Stage 5 — UX Improvements + Final Check (20 min)

Must include at least **one**:

- Retry button on error
- "Back to Home" button
- Display "Completed / Active "
- Limit list to 15 tasks
- Simple UI formatting (basic CSS)

Expected result:

- App is stable, readable, and consistent.

Deliverables (What Students Submit)

Students must submit:

1. Project source code (zip or repository)
2. Screenshots:
   - Home page with loaded tasks
   - Task details page
   - Error state OR loading state (can be simulated)
3. Short explanation (5–8 lines):

- how routing is implemented
- how loading & error are handled

Assessment Criteria (20 points)

Dr. Alla Jammine
alla.jammine@u-paris.fr **Web Programming_2026**

1) Routing & Navigation (5 pts)

- 3 routes implemented correctly (3)
- Navigation via `<Link>` works, no reload (2)

2) API Integration (5 pts)

- Tasks list fetched from API correctly (3)
- Task details fetched by id correctly (2)

3) Loading / Error States (4 pts)

- Proper loading UI (2)
- Proper error UI (2)

4) Code Quality & Architecture (4 pts)

- Separation of concerns (pages/components/services) (2)
- Clean, readable code + no console errors (2)

5) UX / Completeness (2 pts)

- Clickable items, back button, retry, or clean UI (2)

Bonus (+1 max, without exceeding 20):

- Use `AbortController` or request cancellation in `useEffect`
- Use `axios` with interceptors
- Add a small search box for tasks

Instructor Demo Checklist (10 min submission)

Student must demonstrate:

- Navigation between Home / About / Task Details
- Tasks loaded from API
- Loading state displayed (can reload page)
- Error state displayed (simulate by changing URL in api.js)
- Clicking a task opens its details page

Expected UI (Text Spec)

Navbar (always visible):

- Home | About

Home:

- Title + stats
- Tasks list (from API)
- Each item clickable → details

Details:

- Task ID
- Title
- Completed status
- Back to Home

About:

- 4–6 lines: what the app does + tech used